

ETNAS: An energy consumption task-driven neural architecture search

Dong Dong^{a,b}, Hongxu Jiang^{a,b}, Xuekai Wei^{c,*}, Yanfei Song^{a,b}, Xu Zhuang^d, Jason Wang^e^a Beijing Key Laboratory of Digital Media, State Key Lab Virtual Real Technology & Systems, Beihang University, Beijing, 100191, China^b Hangzhou Innovation Institute, Beihang University, Hangzhou, 310051, China^c School of Computer Science, Chongqing University, Chongqing, 400044, China^d Guangdong Opel Mobile Communications Co., Ltd., OPPO, Chengdu, 610000, China^e Guangdong Opel Mobile Communications Co., Ltd., OPPO, Nanjing, 210000, China

ARTICLE INFO

Keywords:

Neural architecture search
 Differentiable architecture search
 Power consumption
 Task-driven
 Multi-objective optimization

ABSTRACT

Neural Architecture Search (NAS) is crucial in the field of sustainable computing as it facilitates the development of highly efficient and effective neural networks. However, it cannot automate the deployment of neural networks to accommodate specific hardware resources and task requirements. This paper introduces ETNAS, which is a hardware-aware multi-objective optimal neural network architecture search algorithm based on the differentiable neural network architecture search method (DARTS). The algorithm searches for a lower-power neural network architecture with guaranteed inference accuracy by modifying the loss function of the differentiable neural network architecture search. We modify the dense network in DARTS to simultaneously search for networks with a lower memory footprint, enabling them to run on memory-constrained edge-end devices. We collected data on the power consumption and time consumption of numerous common operators on FPGA and Domain-Specific Architectures (DSA). The experimental results demonstrate that ETNAS achieves comparable accuracy performance and time efficiency while consuming less power compared to state-of-the-art algorithms, thereby validating its effectiveness in practical applications and contributing to the reduction of carbon emissions in intelligent cyber-physical systems (ICPS) edge computing inference.

1. Introduction

In recent years, with the rapid development of artificial intelligence technology, the design of NAS has become a popular research area. However, since the training of neural networks requires a large amount of computational resources and energy, how to design efficient neural networks while achieving sustainable computing has become a focus of research. Sustainable computing refers to reducing the consumption of computational resources and energy while satisfying computational demands, thus achieving more environmentally friendly and economical computing [1,2]. To address this issue, researchers have proposed some NAS algorithms related to sustainable computing. The core idea of these algorithms is to optimize the computational efficiency of neural networks while designing efficient neural networks, thus reducing the consumption of computational resources and energy [3,4]. One of the commonly used algorithms is the reinforcement learning-based NAS algorithm. This algorithm uses the technique of reinforcement learning to design an intelligent body to search for an efficient neural network structure and to improve the performance and computational efficiency of the neural network through the optimization of the reward function [5,6]. Another NAS algorithm related to sustainable computing is

the hardware-aware NAS-based algorithm. This algorithm designs the optimal neural network structure for a given hardware condition by considering the limitation of hardware resources. For example, when performing neural network computation on an embedded device, the storage and computational resource limitations of the device need to be considered [7], and therefore a neural network structure with optimal performance under these limitations need to be designed [8,9].

Hardware-aware neural network architecture search (HW-NAS) [10] has gained great interest by automating the deployment of deep neural networks on resource-constrained edge devices, and although HW-NAS's are promising in terms of performance, it is very difficult to develop the best HW-NAS algorithms and solutions. HW-NAS has to determine a performance evaluation strategy, and the existing methods generally use The first approach is time-consuming and requires knowledge of how the device is compiled and how to set up the measurement pipeline, while the second poses a significant hurdle for NAS researchers and non-hardware professionals. Another thing that is the same as general-purpose NAS is that it is very difficult to perform performance evaluation tests on HW-NAS due to the large

* Corresponding author.

E-mail addresses: BY1906006@buaa.edu.cn (D. Dong), jianghx@buaa.edu.cn (H. Jiang), xuekaiwei2-c@my.cityu.edu.hk (X. Wei), SYanF@buaa.edu.cn (Y. Song), zhuangxu@oppo.com (X. Zhuang), jason1@oppo.com (J. Wang).

<https://doi.org/10.1016/j.suscom.2023.100926>

Received 27 February 2023; Received in revised form 16 October 2023; Accepted 30 October 2023

Available online 10 November 2023

2210-5379/© 2023 Elsevier Inc. All rights reserved.

amount of computational resources required to calculate the search space, hyperparameters, and hardware device differences [11][12].

“Towards Efficient Hardware-aware Neural Architecture Search” by Cai et al. [13] proposed a new search space design and a hybrid search algorithm, which significantly improved the efficiency of HW-NAS. The proposed method is highly scalable and achieved state-of-the-art performance on various hardware platforms. However, this method requires a large number of GPU devices to achieve high-performance results. “PC-DARTS: Partial Channel Connections for Memory-Efficient Architecture Search” by Chen et al. [14] proposed a memory-efficient approach for HW-NAS by reducing the number of channels in intermediate features. The proposed method achieved state-of-the-art performance with high computational efficiency. But this method may not be suitable for complex models with a large number of parameters.

Through the analysis, we found that the aggregated loss function of the existing neural network architecture search algorithm is more dedicated to achieving higher accuracy while taking into account some other indicators, so it can be hardware-aware of power consumption, but most of them use the loss function multiplied by the proportionality factor and power consumption, so as to achieve the architectural search of power consumption and accuracy trade-off [11]. Since it is difficult to control the actual effect of the scaling factor this search logic does not meet the actual needs in the grounded application [13][15]. We propose an inference metric-dependent neural network architecture search algorithm that can reduce the inference power consumption of edge inference devices while prioritizing the metrics to ensure the completion of the task requirements, thus reasonably releasing the excess computational consumption.

The main contributions of this thesis are as follows.

1. We proposed a hardware-aware neural network architecture search algorithm ETNAS based on task requirements, which modified the logic of NAS search by adding a soft loss to the weight-optimized loss function L_{train} , i.e., searched for a lower-power neural network architecture with guaranteed inference accuracy.

2. We modified the Super-net architecture of differentiable neural network architecture search so that it searched for networks with lower memory footprint at the same time to be suitable for running on generally memory-constrained edge-end devices, in addition to its faster network architecture search and training of weights.

3. We selected common operator types in lightweight network models and measured the power consumption of alternative operators in the field on edge-side devices (e.g., ZCU102, Huawei Atlas200DK) under various parameter settings. Better power consumption performance was achieved in the experiments.

In summary, our proposed method tackles HW-NAS from a novel perspective. We introduce a novel search space that explicitly considers hardware constraints and jointly optimizes for both accuracy and efficiency metrics. By incorporating domain-specific knowledge and leveraging reinforcement learning, our approach effectively explores the trade-off between model performance and hardware requirements, leading to more efficient and effective neural architectures tailored for the specific target hardware. This leads to improved efficiency and effectiveness in settings where hardware is a constraint.

This paper is organized as follows. Section 2 summarizes the related works. Section 3 introduces the core strategy of this method. Then Sections 4 and 5 show the experimental results and discussion. Section 6 summarizes the work of this paper.

2. Related works

Much of the current research has focused on improving the accuracy and efficiency of NAS processes and has included NAS energy consumption as an important consideration, especially in resource-constrained environments. One approach is to reduce the computational cost of

NAS by using low-cost hardware or by employing approximate computing techniques. For example, Luo et al. [16] proposed a hardware-friendly NAS approach based on hardware-aware network transformation, which can significantly reduce the number of convolutional operations while maintaining high accuracy. Meanwhile, He et al. [17] proposed an efficient NAS method that exploits the sparsity of neural networks to reduce the computational cost. Others introduce energy consumption as a constraint in the NAS process. For example, Cai et al. [11] proposed an energy consumption-constrained NAS method that can directly optimize the energy consumption of the target network and achieve better performance in terms of energy efficiency. Similarly, Yu et al. [12] proposed an energy-aware NAS approach that uses hardware cost metrics as part of the fitness function. However, in practice we found that most of the hardware-aware loss functions used in the work are controlled by a fuzzy scaling factor, and it is difficult to obtain a low-power network that meets the practical requirements. Therefore we further modified their weight-optimized loss functions.

A widely used NAS method nowadays is gradient descent-based search for differentiable architectures, where the agent model uses gradient descent to update the network structure. For example, the DARTS algorithm proposed by Liu et al. [18] is a gradient descent-based approach that uses differentiable operations to update the network structure and achieves SOTA performance on multiple datasets. However, the dense network used by their Super-net is too memory intensive for edge-side deployments. For real-world applications of edge-side devices, we improve the connection of the Super-net so that our approach focuses more on lightweight network search and reduces the time and energy consumed for searching and performing inference on the network architecture. While these techniques show promise in model compression, they are limited in their ability to jointly optimize the architecture and hardware-aware criteria, potentially missing out on superior designs. Furthermore, these works incorporate architectural constraints during the search process, aiming to directly optimize for target hardware platforms. However, such methods may face challenges in handling complex and diverse hardware environments and may lack adaptability for future hardware advances.

3. Method

ETNAS architecture search space $\mathcal{O}$ is encoded in a Super-net represented by a directed acyclic graph (DAG). By continuously connecting discrete structures and learning network architecture parameters α from $\mathcal{O}(\alpha^*)$, the optimal network architecture α^* is finally selected from the search space $\mathcal{O}$. The searched network architecture α^* is a subgraph of $\mathcal{O}(\alpha \in \mathcal{O})$, denoted as a weighted network $\mathcal{N}(\alpha, w)$ with weight w .

In Section 3.1, we elaborated on the structural design of DAGs for resource-constrained edge deployment. In Section 3.2, we introduced the optimization of NAS algorithms, which involves optimizing both weight and architecture parameters using a bi-level optimization approach based on the continuous search space. Additionally, in edge deployment environments, there are often high requirements for computation complexity, latency, and power consumption. In Section 3.3, we proposed a loss function that focuses on improving network power consumption as much as possible after achieving a specified accuracy value by incorporating soft losses into the L_{train} loss function. We provided a detailed explanation of this approach.

3.1. Define search space

For the edge deployment environment, our framework needs to search out two kinds of modules as the building blocks of the NAS network, and the learned Cells are applied in the middle and both ends of the network.

Following the concept inherited from DARTS, each cell is a DAG consisting of n nodes. In a convolutional neural network, each node $x^{(i)}$ in the graph represents a feature map, and each directed edge

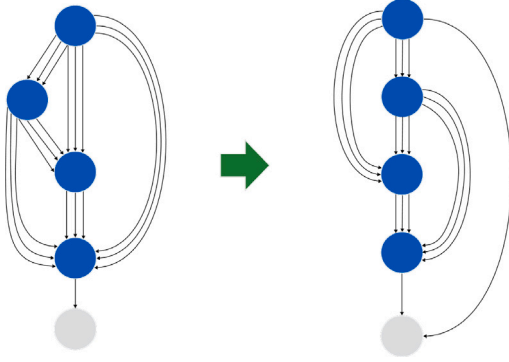


Fig. 1. The cell network designed for edge deployment environment with a search space dimension of 3. Continuous relaxation of the search space by placing a mixture of candidate operations on each edge.

$o^{(i,j)}$ represents an operation between two nodes. An operator $o^{(i,j)}$ is a transformation applied to the feature map of node $x^{(i)}$, which transforms the feature map of node $x^{(i)}$ into the feature map of node $x^{(j)}$. (Eq. (1))

$$x^{(j)} = \sum_{i < j} o^{(i,j)}(x^{(i)}) \quad (1)$$

the input of each cell is set to be the output of all previous nodes. Therefore, the output of the entire cell is obtained by reducing the outputs of all intermediate nodes used.

Through comparative experiments, we found that this approach greatly increases the memory consumption and power consumption of the searched network, which is particularly evident on edge devices. Therefore, in this algorithm, each node in a cell is computed based only on the previous two nodes, and skip connections are used at the beginning and end of the cell. This greatly reduces the instantaneous memory consumption and increases the parameter size and speed that can be run on edge devices (see Fig. 1).

Through comparative experiments, we found that this approach greatly increases the memory consumption and power consumption of the searched network, which is particularly evident on edge devices. Therefore, in this algorithm, each node in a cell is computed based only on the previous two nodes, and skip connections are used at the beginning and end of the cell. This greatly reduces the instantaneous memory consumption and increases the parameter size and speed that can be run on edge devices.

We defined the following operators in the search space $\mathcal{O}$: 3×3 , 5×5 depthwise separable convolution with dilation, 3×3 , 5×5 , 7×7 depthwise separable convolution, 1×1 convolution, 3×3 max pooling, and zero operation. All operators have a default stride of 1, and the feature maps are padded to maintain the resolution. We apply a Relu-Conv-BN operation after each convolution.

We designed two types of cells, which are applied to the side stages and the center stage of the search network respectively. Based on prior experience, the length of the cells in the side stages is set to a fixed 5 nodes, while the length of the cell in the center stage is set to 7 nodes. Each node in the cell is calculated only by the previous two intermediate nodes, and the output node is the result of adding the calculation results of the previous two intermediate nodes and the input node with skip connection, which is then adjusted in feature map size through necessary 1×1 convolution kernels. Except for zero and skip connections, all operations are batch normalized and ReLU-activated.

Parametric Design of the Search Space Based on DAGs and Lightweight Candidate Cells: In this section, we parameterize the search space design based on the DAGs structure and lightweight candidate cells, as shown in Fig. 2. First, we pre-define some fixed blocks at the front and back ends of the network structure based on the input data

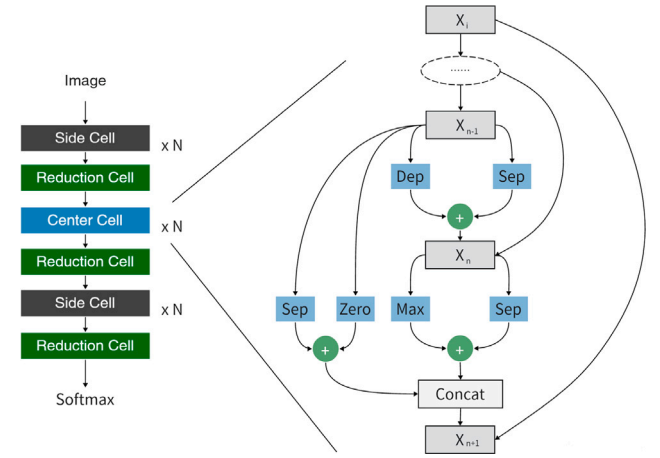


Fig. 2. The predefined network structure, where both the Side Cell and Center Cell are search blocks that are populated by NAS algorithms.

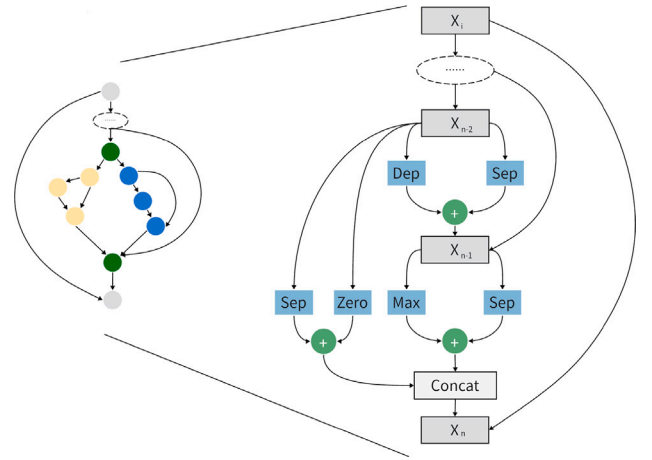


Fig. 3. At the end of the optimization replacing each mixed operation $\bar{o}^{(i,j)}$ with the most likely operation $o^{(i,j)}$, the discrete structure of the cell is obtained.

and configure trainable blocks in three parts. At the same time, the number of trainable blocks needs to be flexibly set according to the specific task requirements. A trainable block is composed of one or more candidate cells.

3.2. Generating and optimizing the computational graph of the target model

$\mathcal{O}$ is a set of candidate operator collections (convolution, pooling, zero operators), and each operator is a function that operates on the input feature map $o(x^{(i)})$. Softmax is applied to all candidate operators to make the search space continuous. (Eq. (2))

$$\bar{o}(x^{(i,j)}) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x) \quad (2)$$

$\mathcal{O} = \{o^1, o^2, \dots, o^n\}$ is a set of candidate operators, where $\alpha_o^{(i,j)}$ represents the weight of operator $o^{(i,j)}$. Specifically, for a pair of nodes (i, j) , all mixed weights of candidate operators become a $|\mathcal{O}|$ -dimensional vector $\alpha^{(i,j)}$ after being passed through Softmax, and these vectors are learned to search for the neural network architecture. After updating these vector parameters, the operator with the highest probability in the vector is selected as the operator between the two nodes, i.e., $o^{(i,j)} = \arg\max_{o \in \mathcal{O}} \alpha_o^{(i,j)}$ (see Fig. 3).

As a DARTS-like algorithm, we optimize the loss function on the validation set using gradient descent to search for the architecture α

and weights w .

$$\begin{aligned} \min_{\alpha} \quad & \mathcal{L}_{val}(w^*(\alpha), \alpha) \\ \text{s.t.} \quad & w^*(\alpha) = \arg \min_w \mathcal{L}_{train}(w, \alpha) \end{aligned} \quad (3)$$

$\mathcal{L}_{train}$ and $\mathcal{L}_{val}$ represent the training and validation losses, respectively. Both losses depend on the architecture α and the weights w in the network, and the neural network architecture search needs to find the α^* that minimizes the validation set loss $\mathcal{L}_{val}(w^*, \alpha^*)$, where w^* is obtained by training the network on the training set. This requires solving a bi-level optimization problem. (Eq. (3))

$$\nabla_{\alpha} \mathcal{L}_{val}(w', \alpha) - \xi \nabla_{\alpha, w}^2 \mathcal{L}_{train}(w, \alpha) \nabla_w \mathcal{L}_{val}(w', \alpha) \quad (4)$$

Because optimizing Eq. (3) directly is difficult, an iterative approximation optimization method is used to simplify it into Eq. (4). A second-order approximation function is used to optimize α and w , where ξ is a hyperparameter, and in the experiments, it is set to 1, which means a second-order approximation function is used. (Eq. (4))

Algorithm 1: Generating and optimizing the computational graph

Input: validation data D_{val} , training data D_{tr} , number of approximation ξ , number of kept path k , max iteration T

- 1 Create a mixed operation $\sigma^{(i,j)}$ parametrized by $\alpha^{(i,j)}$ for each edge $\sigma^{(i,j)}$
- 2 **while** not converged **do**
- 3 **for** $\tau = 1, \dots, T/k$ **do**
- 4 Update architecture α by descending $\nabla_{\alpha} \mathcal{L}_{val}(w - \xi \nabla_w \mathcal{L}_{train}(w, \alpha), \alpha)$ ($\xi = 1$)
- 5 **for** $i = 1, \dots, k$ **do**
- 6 Update weights w by descending $\nabla_w \mathcal{L}_{train}(w, \alpha)$
- 6 Derive the final architecture based on the learned α .

This method can be summarized by the pseudocode (Algorithm 1). Define a set of candidate operators $\mathcal{O}$ as the options in the search space and construct a supergraph, where each node represents an intermediate layer of the neural network, and the connections between nodes represent the relationships between the candidate operators. Initially, the weights of the candidate operators in each connection are assigned the same value. Train the network on the training set D_{tr} and calculate the training error. Compute the gradient of the candidate operators in each connection and use the gradient information to update their weights. Apply the Softmax operation to the updated weights to transform them into a probability distribution, which represents the selection probability of the candidate operators. Evaluate the performance on the validation set D_{val} and calculate the validation error. Use bi-level optimization to update the weights of the neural network structure and the weights of the candidate operators. Repeat the above steps until the search is completed. After the search is completed, select the candidate operator with the highest probability as the final structure of the neural network and train it on the entire dataset.

3.3. Hardware awareness for deployment

In addition to the accuracy of the DNN model, the power consumption of operators during inference on hardware platforms is also an important factor. During the model training process, it is necessary to have differentiable power consumption in order to ensure the continuity of the gradient-based backpropagation algorithm, so that we can optimize the weight parameters successfully. For the power consumption terms, we use a table-based model for efficiency estimation. (Eq. (5))

$$PC(a) = \sum_l \sum_i m_{l,i} \times PC(b_{l,i}) \quad (5)$$

The power consumption of each operator, $PC(b_{l,i})$, is a constant coefficient, so the total architecture power consumption with respect to the mask $m_{l,i}$ is differentiable. Therefore, even after adding hardware power consumption, the above NAS algorithm can still optimize the search results through the backpropagation.

$$\text{maximize}_m \quad \left[\frac{\text{ACC}(m)}{T} \right]^w \times PC(m) \quad w = \begin{cases} \alpha, & \text{if } \text{ACC}(m) \leq T \\ \beta, & \text{otherwise} \end{cases} \quad (6)$$

Finally, we propose an aggregation loss function (Eq. (6)) with power consumption constraints, which transforms the single-objective problem into a multi-objective trade-off optimization problem. We also add a hyperparameter T , where the value of T should be the training accuracy of the deployed target task. This guides the network to focus more on optimizing power consumption when the accuracy is satisfactory. Both α and β are hyperparameters. By adjusting these two hyperparameters, the sensitivity of the loss function to accuracy can be changed. Note that maintaining a small amount of accuracy weight can speed up the convergence of the model. In the experiments in this article, the hyperparameters are set to $\alpha = 2$ and $\beta = -1$.

4. Evaluation

ETNAS algorithm, combined with relevant frameworks such as Tensorflow and Pytorch, can be applied to the actual deployment of neural networks to generate better neural networks on target devices and target tasks, and better match the relationship between tasks and networks to achieve acceleration. For example, MobilenetV2[19], Resnet50[20], YOLOv5[21] and other networks have verified the availability of this algorithm in the actual deployment of hardware devices such as FPGA ZCU102 and Huawei Atlas200DK.

In this article, our algorithm is evaluated in Pytorch (version 1.13.1). The experiment was first conducted on CIFAR-10 [22], which is widely used in image classification and contains 50,000 training samples and 10,000 test samples, with a size of 32×32 . Since the NAS algorithm takes a long time, we only use the official training set for experiments. At the same time, we further carried out a comparative experiment on ImageNet [23], which contains about 1.2 million high-resolution natural images for training, covering 1000 categories of objects. The verification set contains 50k images. Images are adjusted to 224×224 , 512×512 , 1024×1024 before being fed to the network. We adopted Top-1 precision report sheet as the evaluation result.

We utilized a learning rate of 0.1 at the onset of training and gradually applied cosine learning rate decay, reducing it to $1e-4$. To optimize the model, we employed the SGD optimizer with a momentum of 0.9 and weight decay of $1e-4$. Our batch size was 128, and we trained the models for 50 epochs. To prevent overfitting, we implemented the Early Stopping strategy, monitoring the validation performance and stopping training when there was no improvement. Warm-up training was executed during the first five epochs with a learning rate of 0.01, gradually increasing it to the initial learning rate of 0.1. This stabilized the model at the initial training phase. To prevent gradient explosion, we set the gradient clipping threshold to 5.0 during training. Our experimental setup enabled us to successfully conduct the Neural Architecture Search on the CIFAR-10 dataset. Consequently, we obtained an optimized neural network architecture that demonstrated high classification performance on CIFAR-10 and efficient hardware utilization, thereby making it suitable for deployment on the target hardware platform.

In order to ensure the fairness of the comparison, we carried out the experiment under the same hardware and software environment. We use 2 NVIDIA RTX 4090 as the model training platform for our experiments, driven by 16 384 NVIDIA®CUDA®cores running at 2.52 GHz with 24 GB GDDR6X video memory. All models were trained using SGD with a momentum of 0.9 and a learning rate starting at 0.1 and

Table 1
Power and time consumption of lightweight candidate cells.

Input size	Output size	Expansion factor	Kernel size	Stride	Latency (ms)	Ave power (mW)
16 × 16 × 32	16 × 16 × 16	1	3 × 3	1	0.08195	1.99
16 × 16 × 32	16 × 16 × 16	1	5 × 5	1	0.128	2.37
16 × 16 × 32	16 × 16 × 16	1	7 × 7	1	0.155	2.72
16 × 16 × 16	8 × 8 × 24	3	3 × 3	2	0.277	3.91
16 × 16 × 16	8 × 8 × 24	3	5 × 5	2	0.748	9.02
16 × 16 × 16	8 × 8 × 24	3	7 × 7	2	1.577	19.67
16 × 16 × 16	8 × 8 × 24	6	3 × 3	2	0.553	6.75
16 × 16 × 16	8 × 8 × 24	6	5 × 5	2	1.495	18.34
8 × 8 × 24	8 × 8 × 24	3	3 × 3	1	0.407	6.09
8 × 8 × 24	8 × 8 × 24	3	5 × 5	1	1.395	17.27
8 × 8 × 24	8 × 8 × 24	3	7 × 7	1	3.576	45.13
8 × 8 × 24	8 × 8 × 24	6	3 × 3	1	0.813	12.38
8 × 8 × 24	8 × 8 × 24	6	5 × 5	1	2.789	33.92
8 × 8 × 24	8 × 8 × 24	6	7 × 7	1	7.150	87.36
2 × 2 × 192	2 × 2 × 192	6	3 × 3	1	0.968	16.22
2 × 2 × 192	2 × 2 × 192	6	5 × 5	1	5.438	69.05
2 × 2 × 192	2 × 2 × 192	6	7 × 7	1	18.480	236.07

Table 2
Power and time consumption of lightweight subnetworks.

Input size	Type	T	Latency (ms)	Ave power (mW)
16 × 16 × 32	MobileNet	3	0.269	3.84
16 × 16 × 32	MobileNet	6	0.536	7.11
16 × 16 × 32	ShuffleNet	2	0.089	1.64
16 × 16 × 16	ShuffleNet	4	0.088	1.43
16 × 16 × 16	SqueezeNet	0.25	0.052	1.19
16 × 16 × 16	SqueezeNet	0.75	0.022	0.78

Table 3
Performance results on ImageNet classification.

Model	Type	Parameters (M)	FLOPs (M)	Top-1 Acc. (%)	Latency (ms)	Power (w)
MobileNet V2	Manual	3.4	300	71.7	97	1.24
SqueezeNet	Manual	3.2	708	68.3	158	1.49
DARTS	Auto	4.9	595	73.1	211	2.96
NASNet-A	Auto	5.3	564	74.0	204	2.94
ETNAS	Auto	2.4	244	73.1	83	1.13

scaled by 0.1 after 10 epochs. In order to verify the effectiveness of the proposed method on FPGA, it is necessary to first evaluate the forward inference power of lightweight candidate cells. In this paper, the Vivado HLS development tool is used to evaluate the time power consumption of lightweight candidate cells.

When evaluating a subnetwork, we consider its input size, which is the dimensions of the input tensor represented as “height × width × channels”. The type of subnetwork being evaluated is also taken into account. We measure the latency (ms), which is the time it takes for the subnetwork to process a single input in milliseconds. This reflects the inference time and tells us how quickly the subnetwork can make predictions for a given input and T is the hyperparameter mentioned in Eq. (6) of this paper. Through experimental testing, we obtain the optimal network runtime and energy consumption corresponding to different values of T. Additionally, we measure the average power consumption (mW) of the subnetwork during inference, which tells us how much energy is consumed for processing one input.

Specifically, Table 1 shows the real delay and power consumption of a part of candidate cells on FPGA(Xilinx ZCU102 with clock cycle of 100M) under the conditions of input feature graphs, output feature graphs, expansion coefficient, convolution kernel size, and convolution step size with different sizes. The time and power consumption of different subnetworks are collected, and the data is averaged every ten times. We then used the CIFAR-10 dataset for assessment. The power and time consumption of candidate cells was shown in Tables 1 and 2.

To prove the effectiveness of the algorithm, we test our work on ImageNet. Table 3 demonstrates the effectiveness of our method compared with the state-of-the-art manual lightweight networks (MobileNet

Table 4
Average inference performance data for running YOLOv5 on ZCU102.

Model	Board	Scale (w × h)	Inference (FPS)	Performance efficiency (GOPS/W)	Ave power (w)
ETNAS	ZCU102	512 × 512	84	127.9	2.87
NASNet-A	ZCU102	512 × 512	73	114.5	3.99
DARTS	ZCU102	512 × 512	71	98.3	4.36

Table 5
Average inference power consumption of models running on Atlas200DK (in W).

	DARTS (W)	NASNet-A (W)	ETNAS (W)
YOLO-X-nano	4.83	4.55	3.22
YOLO-X-tiny	4.96	4.91	3.13
YOLO-X-s	5.22	4.98	3.91
YOLO-X-x	5.31	4.76	3.75
YOLO-v5-s	5.15	4.81	3.89
YOLO-v5-x	5.10	4.79	3.58
Resnet-50	3.23	3.20	1.92
Squeezenet	2.97	2.83	1.95
MobileNetV2	2.74	2.79	1.44
Shufflenet	2.82	2.91	1.97

V2 [19], ShuffleNet [24] and SqueezeNet [25]), and some well performance auto lightweight networks (DARTS [18] and NASNet-A [26]). The results show that our method is 1.2 times faster and reduces power consumption by about 21% compared to the well-known manual network, and 2.5 times faster and reduces power consumption by about 63% compared to the automatic lightweight network. Meanwhile, our models with smaller FLOPs and parameters.

We changed the size of the pictures and tested the relatively large model (YOLOv5) to calculate the average inference time and energy consumption of each 10,000 pictures, and compared it with the existing methods shown in Table 4.

In our experiment Table 5, we opted for the Atlas200DK of Huawei A310. As part of our methodology, we measured the inference time consumption of 10,000 images with a size of 1024 × 1024 and recorded the average power using official tools. To obtain accurate measurements, we analyzed the profiling data and recorded the time-consuming and power consumption of the model. Our analysis revealed that our method outperforms other models in terms of power optimization.

5. Discussion

Despite the promising advancements in HW-NAS, our proposed method also exhibits certain limitations that warrant consideration. The HW-NAS process typically requires additional hardware profiling and measurements, which can be time-consuming and resource-intensive.

The overhead of obtaining accurate hardware metrics during the search may affect the efficiency gains achieved by the optimized architectures. Finding more efficient and automated ways to profile hardware characteristics will be critical for scalable and practical HW-NAS.

Looking ahead, the field of HW-NAS holds great potential for transformative research and practical applications. To overcome the aforementioned limitations, future efforts should focus on the following directions:

1. **Domain-Generalization Techniques:** Developing domain-generalization techniques that can optimize architectures for a wide range of hardware environments will be crucial. These methods should be capable of learning from a variety of hardware profiles and transferring the knowledge to new hardware platforms without extensive retraining.

2. **Automated Hardware Profiling:** The development of automated and efficient hardware profiling techniques will significantly reduce the overhead associated with HW-NAS. Techniques that can accurately estimate hardware metrics without the need for exhaustive hardware measurements will streamline the optimization process.

3. **Robustness to Noisy Hardware Metrics:** Investigating methods that are robust to noisy or imperfect hardware metrics will ensure the reliability of HW-NAS in real-world scenarios. Techniques that can handle imprecise hardware measurements while still producing high-quality architectures will be valuable.

In conclusion, while HW-NAS faces challenges and limitations, the ongoing research and future outlook offer exciting opportunities to create efficient and adaptable neural architectures optimized for various hardware platforms. By addressing these challenges and exploring new directions, HW-NAS is poised to make significant contributions to the field of efficient and hardware-conscious deep learning models.

6. Conclusion

In this paper, we proposed a hardware-aware multi-objective optimization neural network architecture search algorithm ETNAS based on power task requirements. By modifying the loss function of differentiable neural network architecture search, the algorithm searched for the neural network architecture with lower power consumption under the condition of guaranteeing the inference accuracy. At the same time, we modified the dense network searched by differentiable neural network architecture so that the network searched had a lower memory footprint at the same time to be suitable for running on generally memory-constrained side devices. The experimental results showed that compared with the manual neural network, the proposed method achieved about 1.2 ~ 1.6 times energy consumption reduction on various ICPS edge computing hardware with roughly the same accuracy. Compared with the equivalent automatic lightweight network, the energy consumption was also reduced by about 2 times. The search time of ETNAS was also shorter than that of DARTS. For this purpose, it could automatically search for real-time, accurate, and lower power consumption DNN architectures that were compatible with the hardware, thereby achieving the goal of sustainable computing.

CRedit authorship contribution statement

Dong Dong: Methodology, Software, Writing – original draft. **Hongxu Jiang:** Supervision. **Xuekai Wei:** Writing – review & editing. **Yan-fei Song:** Data curation, Software, Validation. **Xu Zhuang:** Data curation, Software, Validation. **Jason Wang:** Data curation, Software, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

We are grateful to anonymous reviewers for their constructive comments. This work was supported by the National Key Research and Development Program of China (No. 2021ZD0110202) and by the Guangdong Oppo Mobile Telecommunications Corporation Ltd. (No. H20221694).

References

- [1] Junlong Zhou, Yufan Shen, Liying Li, Cheng Zhuo, Mingsong Chen, Swarm intelligence based task scheduling for enhancing security for IoT devices, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* (2022).
- [2] Ashish Mohan Yadav, S.C. Sharma, Cooperative task scheduling secured with blockchain in sustainable mobile edge computing, *Sustain. Comput.: Inform. Syst.* (2023) 100843.
- [3] Junlong Zhou, Liying Li, Ahmadreza Vajdi, Xiumin Zhou, Zebin Wu, Temperature-constrained reliability optimization of industrial cyber-physical systems using machine learning and feedback control, *IEEE Trans. Autom. Sci. Eng.* (2021).
- [4] Wei Jin, Bing Zhou, Sara A. Althubiti, Theyab R. Alsenani, Mohamed E. Ghoneim, Transient stability assessment of power systems using support vector regressor and convolution neural network, *Sustain. Comput.: Inform. Syst.* 37 (2023) 100826.
- [5] Junlong Zhou, Tian Wang, Weiming Jiang, Hongxia Chai, Zebin Wu, Decomposed task scheduling for security-critical mobile cyber-physical systems, *IEEE Internet Things J.* 9 (22) (2021) 22280–22290.
- [6] Tanmay Kumar Behera, Sambit Bakshi, Pankaj Kumar Sa, A lightweight deep learning architecture for vegetation segmentation using UAV-captured aerial images, *Sustain. Comput.: Inform. Syst.* 37 (2023) 100841.
- [7] Liying Li, Yinghui Wang, Haizhou Wang, Shiyuan Hu, Tongquan Wei, An efficient architecture for imputing distributed datasets of IoT networks, *IEEE Internet Things J.* (2023).
- [8] Junlong Zhou, Kun Cao, Xiumin Zhou, Mingsong Chen, Tongquan Wei, Shiyuan Hu, Throughput-conscious energy allocation and reliability-aware task assignment for renewable powered in-situ server systems, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 41 (3) (2021) 516–529.
- [9] Junlong Zhou, Mingyue Zhang, Jin Sun, Tian Wang, Xiumin Zhou, Shiyuan Hu, DRHEFT: Deadline-constrained reliability-aware HEFT algorithm for real-time heterogeneous MPSoC systems, *IEEE Trans. Reliab.* 71 (1) (2020) 178–189.
- [10] Chaojian Li, Zhongzhi Yu, Yonggan Fu, Yongnan Zhang, Yang Zhao, Haoran You, Qixuan Yu, Yue Wang, Yingyan Lin, Hw-nas-bench: Hardware-aware neural architecture search benchmark, 2021, arXiv preprint [arXiv:2103.10584](https://arxiv.org/abs/2103.10584).
- [11] Han Cai, Ligeng Zhu, Song Han, Proxylessnas: Direct neural architecture search on target task and hardware, 2018, arXiv preprint [arXiv:1812.00332](https://arxiv.org/abs/1812.00332).
- [12] Jiaheng Liu, Shunfeng Zhou, Yichao Wu, Ken Chen, Wanli Ouyang, Dong Xu, Block proposal neural architecture search, *IEEE Trans. Image Process.* 30 (2020) 15–25.
- [13] Hadjer Benmezziane, Kaoutar El Maghraoui, Hamza Ouarnoughi, Smail Niar, Martin Wistuba, Naigang Wang, Hardware-aware neural architecture search: Survey and taxonomy, in: *IJCAI*, 2021, pp. 4322–4329.
- [14] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Guo-Jun Qi, Qi Tian, Hongkai Xiong, Pc-darts: Partial channel connections for memory-efficient architecture search, 2019, arXiv preprint [arXiv:1907.05737](https://arxiv.org/abs/1907.05737).
- [15] Nilotpal Sinha, Kuan-Wen Chen, Neural architecture search using progressive evolution, 2022.
- [16] Hadjer Benmezziane, Kaoutar El Maghraoui, Hamza Ouarnoughi, Smail Niar, Martin Wistuba, Naigang Wang, A comprehensive survey on hardware-aware neural architecture search, 2021, arXiv preprint [arXiv:2101.09336](https://arxiv.org/abs/2101.09336).
- [17] Yihui He, Ji Lin, Zhijian Liu, Hanrui Wang, Li-Jia Li, Song Han, Amc: Automl for model compression and acceleration on mobile devices, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 784–800.
- [18] Hanxiao Liu, Karen Simonyan, Yiming Yang, Darts: Differentiable architecture search, 2018, arXiv preprint [arXiv:1806.09055](https://arxiv.org/abs/1806.09055).
- [19] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, Liang-Chieh Chen, Mobilenetv2: Inverted residuals and linear bottlenecks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.
- [20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [21] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi, You only look once: Unified, real-time object detection, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

- [22] Alex Krizhevsky, Geoff Hinton, Convolutional deep belief networks on cifar-10, Unpubl. Manusc. 40 (7) (2010) 1–9.
- [23] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al., Imagenet large scale visual recognition challenge, *Int. J. Comput. Vis.* 115 (2015) 211–252.
- [24] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, Jian Sun, Shufflenet: An extremely efficient convolutional neural network for mobile devices, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 6848–6856.
- [25] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, Kurt Keutzer, SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 MB model size, 2016, arXiv preprint [arXiv:1602.07360](https://arxiv.org/abs/1602.07360).
- [26] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, Quoc V. Le, Learning transferable architectures for scalable image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8697–8710.